Heat Is a Hidden Accuracy Cost: Pricing a Camera Drone's Power in Streaming Perception

Utkarsh Sengar
Department of Computer Science & Engineering, Indian Institute of Technology Dharwad, India
cs23bt026@iitdh.ac.in

***Abstract*—A camera drone that detects objects in its video chooses, for every frame, whether to run the detector on its own computer, on an edge drone or on a ground GPU. On a fanless drone computer that choice has a cost a scheduler does not see: heat. We measure it on a real three-tier system (a Raspberry Pi 5 camera drone, a Jetson Orin Nano edge drone and a ground GPU, over Wi-Fi and a batman-adv mesh, scored with streaming AP on VisDrone video). Onboard detection keeps the board throttled most of the time, and throttling alone costs onboard streaming AP 37–59 % against the same runs at the full clock. A controller that prices the camera drone's measured power inside a streaming-accuracy objective, with online dual variables for power and freshness, removes most of this cost without losing accuracy: on Wi-Fi it raised streaming AP by +0.0074±0.0030 (16 %, 20 pairs) and +0.0056 (12 %) in a second campaign; on the mesh, removing the prices left accuracy almost unchanged but cost 1.47 W and 13.8 °C and kept the board throttled 39 % of the time instead of 0 %. It does not remove all of it: on 24-minute Wi-Fi missions with total blackouts the board still spent 19–27 % of the time at or above 80 °C, which the replay traces to the controller's blackout hold. In a trace replay validated against the hardware, the greedy priced controller reaches 97–100 % of a per-phase hindsight oracle. An AccuMO-style 30-frame planner is slightly ahead of it there, but that lead lies within the replay's known bias. The price update carries a sample-path guarantee: mean power stays below the budget plus a price term and a hold-and-cap term. All 66,655 logged updates of the 376 priced runs follow the coded rule, so the bound holds on every run (an excess of at most 0.07 W on the mesh and 0.15 W over 24 minutes, where the price sat at its cap). Through a measured thermal model the budget sets the board's mean temperature (within 1.6 °C). Its objective must use the decay curve of the forecaster that runs: with the curve without forecasting it loses 0.0059 sAP.**

I. Introduction

Object detection on a drone has three places to run: the drone's own computer, an edge drone that flies with it, and a ground station. Offloading and per-frame adaptation are well studied for phones and cameras [1]–[3], streaming perception defines how to score a detector whose results arrive late [4], [5], and multi-hop UAV offloading has been built on testbeds [6]. Offloading schedulers usually treat the drone's own computer as a server with a fixed speed; thermal management of on-device inference [7] is studied separately from where each frame should go. A small camera drone carries a fanless board, however, and running the detector on it heats the board until its firmware lowers the clock: the onboard server slows down, and every other task on the board with it.

Thesis. On a fanless drone computer, heat is a hidden accuracy cost, and a scheduler that prices the board's measured power inside its accuracy objective removes most of it without giving up accuracy. We support this on real hardware, and we report where it falls short:

- *Heat costs accuracy.* Running YOLOv8s onboard kept the Pi throttled 64 % of short runs that started cool and 89 % of 24-minute runs; throttling alone cost 37–59 % of onboard streaming AP against an emulated full clock (Section VII-A).
- *Pricing measured power removes the cost.* A dual-ascent price on the board's measured power, inside a streaming-accuracy objective, raised streaming AP on Wi-Fi (+0.0074 and +0.0056 in two campaigns of 20 pairs; +0.0189 and +0.0165 with the standard metric, Holm $p = 0.002$ and 0.011) and on the mesh removed throttling at the same accuracy (Section VII-B); relative to the unpriced controller the Wi-Fi gains are 16 % and 12 %. On 24-minute Wi-Fi missions with total blackouts the board was still at or above 80 °C 19–27 % of the time (Section V).
- *The price has guarantees.* Mean power stays below the budget plus two named terms on every run: every one of the 66,655 logged price updates of the 376 runs follows the coded rule. The budget sets the mean temperature, and an idealised version is within a small, stated gap of the best power-feasible policy (Section V).
- *The greedy priced choice is near the best schedule we can construct* (trace replay): 97–100 % of a per-phase hindsight oracle, no gain from a clairvoyant link, and within the replay's bias of an AccuMO-style planner [8]; on hardware it is 0.0112–0.0278 sAP ahead of re-implemented literature schedulers (Section VII-C).
- *The objective must match the forecaster*: valuing a result by the decay measured *after* motion forecasting beats the curve without forecasting and a flat one (Section VII-D).

Supporting results: the best place to decide depends on the mesh topology (Section VII-E); a first two-camera hardware swarm and a calibrated swarm simulation (Section VII-F); and negative results, a predictive heat price adds nothing on top of the energy price and does worse in its place, and pricing the edge drone's energy saves little (Section VII-G).

II. Related Work

Offloading and adaptation. Odessa [1] adapts offloading and parallelism for interactive perception, Glimpse [2] tracks objects on the device to hide offloading delay, and Elf [3] splits frames across servers; configuration and bandwidth adaptation for video analytics is studied in Chameleon [9], AWStream [10] and [11], [12]. Detection-specific offloading triggers use confidence (OffloaD [13]) or an estimated offloading reward [14]; we re-implement both as baselines. **Streaming perception.** Li et al. [4] score a detector by the accuracy of the result available at each frame; StreamYOLO [5] and Chanakya [15] improve or adapt the detector, and StreamFlow applies optical-flow forecasting to UAV video [16]. We score a distributed offloading system with it. **Accuracy-aware scheduling.** AccuMO [8] predicts how accuracy decays between offloads and plans over a horizon; AoPI [17] optimises freshness and accuracy of video analytics at the edge. Neither prices energy or heat. LEAF [18] trades energy against accuracy with a fixed weight; energy-aware DNN offloading and offloading under energy or latency budgets are studied in [19]–[21], and quality-aware offloading for cooperative perception in [22]; Phoenix [7] manages on-device inference under thermal limits. **Heat on edge devices.** Throttling cuts the throughput of continuous inference on a Raspberry Pi by up to about 90 % without cooling [23]; thermal-aware schedulers choose between on-device processors [24], and a threshold policy jointly sets offloading, DVFS and model size under heat and memory limits [25]. These measure frame rate or latency on one device; we measure the streaming-accuracy cost of heat in a distributed offloading system and price it with a guarantee. Freshness under an energy budget is also studied as age of information, e.g. with a safe DQN for UAV-aided edge computing [26]; our freshness price plays that role inside an accuracy objective. **Why a few watts matter.** The camera board draws a few watts against the motors' far larger flight power; on drones, compute matters through mission time [27], and here through heat, which the board must shed without a fan. **UAV offloading.** A^2-UAV [6] optimises DNN offloading over a multi-hop UAV testbed; Qu et al. [28] orchestrate video analytics across a drone network edge; multi-UAV edge computing is mostly studied analytically [29], [30]. **Split computing.** Splitting the detector between drone and server [31], [32] and learned feature compression [33], [34] are alternatives to sending the image; on our hardware the image is the better payload (companion study), so the controller sends JPEG frames.

III. System

Tiers and links (Fig. 1). The camera drone is a Raspberry Pi 5 without a fan (YOLOv8s: 758 ms per frame once it throttles); the edge drone is a Jetson Orin Nano (22.7 ms, FP16); the ground station is a laptop GPU. The Pi reaches the Jetson and the ground over Wi-Fi or over an ad-hoc batman-adv mesh, on which the route to the ground can be forced through the Jetson (multihop). Link faults (rate limits, loss, blackouts) are applied per phase with `netem` on the

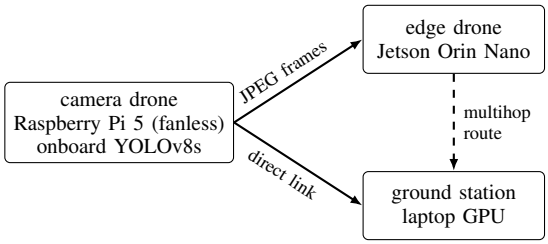


Fig. 1. Testbed. Every frame is detected onboard or sent as a JPEG to the edge drone or the ground station, over Wi-Fi or a batman-adv mesh; in the multihop topology the direct link is blocked and the route crosses the edge drone. Results return to the camera drone.

camera drone. **Workload and metric.** The camera streams the VisDrone2019-VID validation clips [35] at 30 frames/s. Detectors are YOLOv8 [36] models (onboard: YOLOv8s at 320 or 640 px; servers: YOLOv8s at 320–640 px and YOLOv8m at 640 px, JPEG quality 90 or 50). The Wi-Fi campaigns ran COCO-trained weights and are also re-scored with the detections of the same models fine-tuned on VisDrone (reported as "standard metric"); the mesh campaigns and the onboard-only runs ran the VisDrone-trained weights. Accuracy is streaming AP (sAP): at every frame the newest result that has arrived is scored against that frame's labels, with a camera-motion-aligned forecaster that moves stale boxes forward, running live on the Pi.

IV. Controller

Every camera tick each idle server s (the edge drone, the ground station, or the camera's own detector) proposes its best setting v, and the frame goes to the idle server with the largest value if that value is positive; otherwise the frame is skipped. The value of a result is the accuracy it provides while it is held:

$$V(s,v) = \frac{A_v}{A_{\text{ref}}}\, H(L,T)\, p_{\text{late}} - \mu\, E_{\text{cam}}(s,v) + \lambda_F\, p_{\text{on}}\, \mathbf{1}[\text{AoI} + L \geq D], \tag{1}$$

where A_v is the measured accuracy of setting v, L the expected latency (encode, transfer at the live goodput estimate, server time, round trip), T how long the result will be held (the server's busy time per frame divided by the number of servers that can currently deliver), $H(L,T)$ the mean of the measured decay $\rho(\cdot)$ of streaming accuracy over the holding window (sampled at L, $L + T/4$, …, $L + T$), p_{on} and p_{late} the probabilities of arriving within the 300 ms deadline D and a 1 s late window under Gaussian goodput uncertainty (for onboard inference both are 0 or 1 from its measured time, and the factor p_{late} is 1), and E_{cam} the camera drone's modelled energy for the frame from measured constants (onboard inference, or encoding and radio). The decay ρ is measured on the training clips with the forecaster that runs (Fig. 2).

The energy price follows the measured board power $\bar{P}$ of the camera drone's board (mean of the last second): $\mu \leftarrow \min(5, \max(0, \mu + 0.1(\bar{P} - B)))$ once per second, with budget $B = 2.8$ W (3.5–3.6 W in 40 runs of two early Wi-Fi

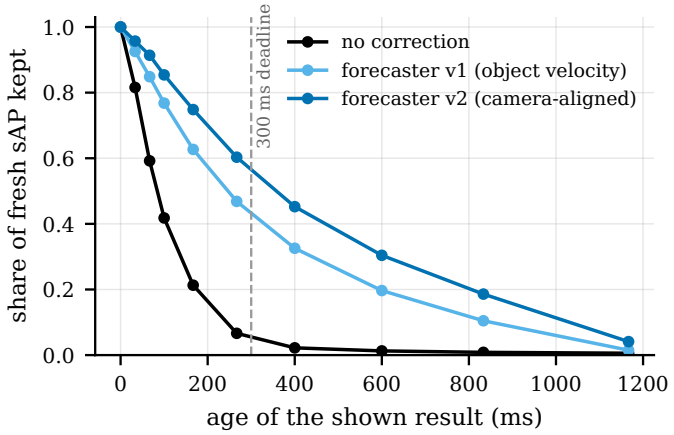


Fig. 2. Streaming accuracy kept by a result shown late (VisDrone-VID train clips): without correction, with object velocity forecasting, and with camera-aligned forecasting. The controller's objective uses the curve of the forecaster that runs.

campaigns). Two exceptions: at a measured 78 °C or more, μ is at least 1 (a thermal guard); and while, for every link, the setting with the highest value has at most a 20 % chance of delivering in time (and the board is below 85 °C), onboard inference is not priced and μ is held, also for 1 s after (anti-windup: the budget cannot be met then). The freshness price follows the share $\hat{F}$ of the last 5 s with a result younger than D: $\lambda_F \leftarrow \min(2, \max(0, \lambda_F + 2(0.95 - \hat{F})))$, once per second. The edge-priced variant subtracts $\mu_e E_{\text{edge}}$ for the Jetson's energy, with μ_e following the Jetson's measured extra power against its own budget in the same way. The heat price λ_T follows a measured thermal model's 60 s prediction $\hat{\theta}$ of the board temperature: $\lambda_T \leftarrow \min(2, \max(0, \lambda_T + 0.01(\hat{\theta} - 75\,^{\circ}\text{C})))$; it was tested in two ways: on top of the energy price (camera joules priced at $\mu + \lambda_T$; Wi-Fi) and instead of it (λ_T alone, no energy price and no exceptions; mesh). Every logged update of the three other prices follows these rules (`price_rules_check.py`): 53,758 updates of λ_F in 158 runs, 34,929 of λ_T in 170 and 43,355 of μ_e in 239, with 0, 0 and 0 exceptions. The caps on λ_F and μ_e were added on 13 October; in 8 and 28 runs of the two Wi-Fi campaigns before that, the same rules ran uncapped, and the prices reached 4.1 and 6.4.

V. Guarantees

The price update is a projected dual-ascent step on the power constraint $\bar{P} \leq B$, and the standard virtual-queue argument [37] gives a bound on every run as coded; what is specific here is the explicit term for the hold and the cap, and that the logged runs can be checked against it. Let μ_t be the energy price after the t-th update ($\mu_0 = 0$), P_t the 1-s mean power that update used, $\eta = 0.1$ the step, and $s_t = \mu_t - \mu_{t-1} - \eta(P_t - B)$ the amount by which the update departs from a plain step.

TABLE I
Theorem 1 on every priced run: largest excess of mean power over the budget, mean share of updates held and at the cap, the largest hold and cap terms of the bound, and the largest price reached. †Before the cap and the hold were added.

Campaign	Runs	Excess W	Held %	Cap %	Hold W	Cap W	$\max \mu$
Wi-Fi, main†	120	1.28	0.2	0.0	0.00	0.00	14.1
Wi-Fi, edge price†	80	1.29	0.2	0.0	0.00	0.00	14.2
Wi-Fi, energy†	40	0.92	0.5	0.0	0.00	0.00	10.2
Wi-Fi, B	9	0.66	6.2	0.0	0.48	0.00	4.5
Wi-Fi, table	60	1.06	15.6	0.5	0.74	0.29	5.0
Wi-Fi, 24 min	8	0.55	15.8	19.0	0.42	0.48	5.0
mesh 1	12	0.07	0.2	0.0	0.00	0.00	3.3
mesh 2	21	0.07	0.1	0.0	0.00	0.00	2.9
mesh 3	18	0.04	0.5	0.0	0.00	0.00	3.3
mesh, 24 min	8	0.15	0.2	18.3	0.00	0.11	5.0

Theorem 1 (Power budget, every run)**.** *Let $\mathcal{H}$ be the updates during which the price is held or at its cap. After N updates,*

$$\frac{1}{N}\sum_{t=1}^{N} P_t \;\leq\; B + \frac{\mu_N}{\eta N} + \frac{1}{\eta N}\sum_{t\in\mathcal{H}} (-s_t)^{+}.$$

Proof. Outside $\mathcal{H}$ an update is a plain step ($s_t = 0$), is clipped at zero, or is raised by the thermal guard, so $s_t \geq 0$. Summing the definition of s_t gives $\mu_N - \mu_0 = \eta\sum_t (P_t - B) + \sum_t s_t$, so $\eta\sum_t (P_t - B) = \mu_N - \sum_t s_t \leq \mu_N + \sum_{t\in\mathcal{H}} (-s_t)^{+}$. □

The bound names the only ways to exceed the budget: a high final price (at most $5/(\eta N)$, i.e. 0.21 W over a 4-minute run and 0.035 W over 24 minutes), or updates in $\mathcal{H}$. The smaller η, the larger the price term; Theorem 2 shows the opposite trade for accuracy. Since the inequality follows from the update rule, what can fail on a real run is the rule itself. We therefore classified each of the 66,655 logged updates of the 376 priced runs (10 campaigns) from the logged values alone, as a plain step, clipped at zero, raised by the guard, held (price unchanged), or at the cap (where a hold and a clip look the same). 0 updates fall outside these cases, so the bound holds on every run.

The bound is a decomposition, not a cap: 364 of the 376 runs averaged above their budget. Table I gives the terms by campaign.

- **Mesh** (51 runs): the excess was at most 0.07 W.
- **24-minute mesh missions:** the excess reached 0.15 W. The price sat at its cap for 18 % of the updates, and the cap term accounts for up to 0.11 W of the excess (the hold for 0.00 W).
- **Wi-Fi scenarios with total blackouts:** the excess reached 1.06 W, of which up to 0.74 W comes from the hold. In a blackout, onboard inference is the only way to show results, and the hold lets the board spend power on it instead of winding the price up.
- **Wi-Fi campaigns run before the cap and the hold were added** (marked †): they exceeded the budget by up to 1.3 W, with the price wound up to 14.

Theorem 2 (Accuracy gap, idealised)**.** *Assume that the link and content states are independent and identically distributed*

over one-second slots, that in every slot the controller chooses the action maximising its expected value U minus μ_{t-1} times the true power, with no freshness term, and that every update is a plain projected step (never held, raised by the guard or capped). Let U^* be the best expected value of any policy that knows the states' distribution and meets $\mathbb{E}[P] \le B$. Then

$$\frac{1}{T}\sum_{t=1}^{T}\mathbb{E}[U_t] \;\ge\; U^* - \frac{\eta}{2}\,\frac{1}{T}\sum_{t=1}^{T}\mathbb{E}\big[(P_t - B)^2\big].$$

Proof sketch. This is the drift-plus-penalty argument [37]: with $L_t = \mu_t^2/2$, projection gives $L_t - L_{t-1} \le \eta\mu_{t-1}(P_t - B) + \eta^2(P_t - B)^2/2$; the greedy choice makes $\mathbb{E}[\mu_{t-1}P_t - U_t]$ no larger than under the optimal stationary policy, whose mean power is at most B; summing, with $L_0 = 0$ and $L_T \ge 0$, gives the bound. □

This is a direct application of drift-plus-penalty, with its usual trade: a gap of order η against a price term of order $1/\eta$ in Theorem 1. Theorem 2 is a guide, not a measurement, because the hardware breaks its assumptions in six ways.

- **Not every step is plain.** The price reached its cap in 15 runs, including all 8 24-minute mesh missions, and it was held during blackouts (Table I).
- **The state depends on past actions.** Which servers are busy depends on earlier choices, and so does the board temperature ($\tau = 149$ s), which changes onboard time and power; that dependence is this paper's thesis.
- **The state is correlated in time.** Link phases and video content are Markov, not independent, which adds a mixing term to the bound [37].
- **The choice is per frame.** Decisions are greedy among the idle servers, using modelled per-frame energy, while the price follows the measured board power.
- **The freshness bonus** is a heuristic term.
- **The metric is streaming AP,** not the controller's value.

Evaluated with the measured $(P_t - B)^2$ of the 42 v4.1 mesh runs (median 0.70 W^2, at most 1.6), which belong to the real controller rather than the idealised one, the bound is 0.035–0.08 value units per second, at most 0.3 % of the largest value rate (one full-accuracy result per frame). The empirical counterpart is Section VII-C. In the validated replay, v4.1 reaches 97–100 % of a per-phase oracle that picks, with hindsight, the best of v4.1 and 17 fixed settings in each phase (88 % when faults change every 4 s). This oracle contains v4.1, so it bounds what a better per-phase choice could add, not the best possible schedule.

Proposition 1 (Budget and temperature)**.** *If the board temperature follows $\dot\theta = (a + bP - \theta)/\tau$, its mean over a window of length W is $\bar\theta = a + b\bar P - \tau(\theta(W) - \theta(0))/W$. With Theorem 1, the mean temperature is at most $a + bB$ plus b times the price and hold-and-cap terms, plus a transient that vanishes as W/τ grows. This bounds the mean, not the peak.*

Proof. Integrate the differential equation over the window and divide by W. □

The proposition is a one-line consequence of a standard first-order model; its use is the rule for choosing B. The 6 measured steady levels below 84 °C give $a = 34.4$ °C and $b = 12.7$ °C/W ($\tau = 149$ s). The hottest of them, 83.8 °C, already ran at a reduced clock (2070 of 2400 MHz). On the second half of all 16 runs of 20 minutes or more, $a + b\bar P$ predicts the mean temperature within 1.6 °C on average (at most 2.8). The error is a campaign offset, not scatter: +1.63 °C on the 24-minute Wi-Fi runs and −0.04 °C on the mesh (mean +0.8). The budget therefore sets the mean temperature: 2.8 W predicts 70 °C, and keeping the mean below 80 °C, where the board starts to throttle, needs $B \le 3.6$ W (3.8 W with the fit the controller uses, levels below 80 °C). Peaks and the onset of throttling are not bounded by this rule.

Peaks need a different guard. Under v4.1's guard, a price of at least 1 makes onboard inference costlier, but it can still be chosen. A hard guard forbids it: no onboard inference starts while $\theta \ge \theta_g$.

Theorem 3 (Peak temperature under a hard guard)**.** *In Proposition 1's model, write the power as $P = P_n + P_i$, where $P_i \ge 0$ is the extra power of onboard inference. Let θ_n be the temperature the model gives with P_n alone, from the same start, and suppose $\theta_{\rm lo} \le \theta_n \le \theta_f$ and $P_n \le \hat P$ throughout.*

(a) *Under any schedule, $\theta \ge \theta_n$.*

(b) *Suppose that inferences run one at a time, that none starts while $\theta \ge \theta_g$, that each lasts at most e with $P_i \le \hat P_i$, that $H = a + b\hat P > \theta_g$, and that $\theta(0) \le \theta_g$. Then*

$$\theta \le \theta_g + b\hat P_i\big(1 - e^{-e/\tau}\big) + \frac{(\theta_f - \theta_{\rm lo})(H - \theta_g)}{H - \theta_{\rm lo}}.$$

Proof. The model is linear, so $\theta = \theta_n + \theta_i$, where $\theta_i \ge 0$ solves $\dot\theta_i = (bP_i - \theta_i)/\tau$ from $\theta_i(0) = 0$; this gives (a). For (b), fix t. If no inference started before t, then $\theta(t) = \theta_n(t) \le \theta_f$, which is below the bound (the bound minus θ_f is at least $(H - \theta_f)(\theta_g - \theta_{\rm lo})/(H - \theta_{\rm lo}) \ge 0$, because $\theta_{\rm lo} \le \theta(0) \le \theta_g < H$ and so $\theta_n \le H$). Otherwise let s be the last start and $x = e^{-(t-s)/\tau}$. At s we had $\theta(s) < \theta_g$, so $\theta_i(s) < \theta_g - \theta_n(s)$. After s, only the inference started at s adds heat, for at most e, so $\theta_i(t) \le \theta_i(s)x + b\hat P_i(1 - e^{-e/\tau})$. Two bounds on $\theta_n(t)$ follow from the model: $\theta_n(t) \le \theta_f$, and $\theta_n(t) \le \theta_n(s)x + (1-x)H$. Writing δ_i for the inference term, the second gives (i) $\theta(t) \le \theta_g x + (1-x)H + \delta_i$, and the first gives (ii) $\theta(t) \le \theta_f + (\theta_g - \theta_{\rm lo})x + \delta_i$. As x grows, (i) falls and (ii) rises, so their minimum is largest where they meet, at $x = (H - \theta_f)/(H - \theta_{\rm lo})$. Its value there is the bound. □

The two added terms are the inference already running when the guard trips (δ_i), and radio bursts that push the power above its mean afterwards. Part (a) is the converse. A room warmer by Δ raises θ_n, $\theta_{\rm lo}$, θ_f and H by Δ. Once $\theta_f + \Delta > \theta_g$, no controller keeps the board below θ_g, since the load that remains without onboard inference already heats it more.

On the hardware, the floor is measured directly by the 62 offload-only runs (no onboard inference). Their 1-s mean power averages 2.52 W and reaches $\hat P = 4.05$ W. Their 2 24-minute runs peak at $\theta_f = 67.2$ °C after the first 2τ, with a

second-half mean of at most 61.8. The shorter runs start as hot as the previous run left the board (up to 77.7), which is why (b) assumes the start. With the controller's fit, $\theta_g = 78$ °C, $\theta_{\text{lo}} = a + bP_{\text{idle}} = 58.7$, $\hat{P}_i = 3.1$ W and $e = 0.76$ s (the throttled full-model time), the terms are $\delta_i = 0.18$ and 1.8 °C. The bound is then 79.9 °C in the lab: just below the 80 °C mark. It stays below the 85 °C throttling limit for a room up to 14.5 °C warmer. By (a), no controller can hold 78 °C in a room more than 10.8 °C warmer (16.2 on the floor's mean). The hard guard has not been run on the board; the replay check follows.

The settings, swept in the replay. We swept the step size, the budget, the room temperature and the hold in the trace replay (Section VI). Each change is to one setting of v4.1, on the same seeds and link draws. There are 6 step sizes from 0.02 to 1 and budgets from 2.4 to 4.5 W, on 10 seeds of the 2-minute scenarios and 5 of the 24-minute mission: 285 runs. Every prediction was committed before its runs. Theorem 1 holds in every run. Against the hardware, the replay reproduces mean power ($r = 0.99$) and how the share of time at or above 80 °C varies between policies ($r = 0.98$). The level of that share is off by 16 points on average, so the shares below are comparisons between settings, not predictions of the board's. Its streaming AP is reliable only to about +0.0053, and every accuracy difference below is smaller than that, so we report them without drawing conclusions. Its thermal plant is Proposition 1's own model, so its mean temperatures are no evidence for the proposition.

- **Step size, 2-minute runs.** As η grows, the price term of Theorem 1 falls from 0.41 to 0.017 W on the mission (0.48 to 0.024 W with random faults). The excess over the budget falls with it, from 0.79 to 0.52 W (0.52 to 0.16 W). Theorem 2's gap term grows to 0.55 (0.20) value units per second, but streaming AP does not fall: at $\eta = 1$ it differs from v4.1 by +0.0009 ± 0.0010 (+0.0014 ± 0.0008). We had predicted a loss; the idealised $O(\eta)$ gap is not visible in this system.
- **Step size, 24-minute missions.** The price term all but vanishes (0.13 to 0.001 W), and the excess stays between 0.54 and 0.57 W at every step size. Almost all of it is the hold term (up to 0.44 W): over a long run the step size hardly matters.
- **Budget.** From 2.4 to 4.5 W the mean temperature rises from 73.2 to 78.8 °C, below 80 °C as the proposition says. Yet the board spends 18–21 % of the mission at or above 80 °C at every budget; we had predicted almost no such time below 3.2 W.
- **The hold.** During a blackout, the hold lets onboard inference run unpriced. Switching it off (a diagnostic, not a proposed controller) cuts that time from 18 % to 0 % and the excess from 0.52 to 0.18 W; streaming AP changes by −0.0002 ± 0.0004. On a long mission with blackouts, the hold, not the budget, is what heats the board.
- **A hold capped at 78 °C,** the thermal guard's threshold, keeps the hold for a cool board. It brings the time at or above 80 °C to 0 % and the excess to 0.22 W, with streaming AP +0.0000 ± 0.0001 against v4.1.
- **A hotter room.** This one is an extrapolation: ambient was not logged, so the plant's intercept was shifted by +5 and +10 °C. With the lab's 3.6 W, the time at or above 80 °C rises to 38 % and 60 %. Lowering the budget by $1/b$ per degree, as the proposition suggests, leaves it at 37 % and 57 %. The capped hold does not help either at +10 °C (57 %). There the price is at its cap in 29 % of the updates, and the cap term reaches 0.75 W: the cap is the remaining lever.
- **The cap, and a budget that follows the room,** at +10 °C. Raising the cap from 5 to 10, 20 or 50 gives the same runs, since the price never passes 7.0: the time at or above 80 °C falls to 42 % and freshness to 89.4 %. The rest is the budget: 2.8 W is too much in that room. A budget set from an online estimate of the room's offset (the controller's fit inverted, $\hat{a} = \theta + \tau\dot{\theta} - b\bar{P}$) stays near 2.8 W in the lab (2.65 W, no hot time) and falls in the hot room (a mean of 2.56 W). Under the cap of 5, however, it cannot be enforced: the price sits at the cap in 74 % of the updates, and the board is hot 52 % of the time. With both changes the hot time is 0 % at +5 °C and 0 % at +10 °C (2.63 W). Freshness falls to 94.3 % and 88.8 %, and streaming AP changes by −0.0016 ± 0.0015.
- **The hard guard** (Theorem 3), with the hold capped at 78 °C. The peak is 78.0 °C in the lab and 78.0 °C at +5 °C, and there is no time at or above 80 °C up to +10 °C. At +15 °C the floor alone averages 79.9 °C, above the guard. The board is then hot 84 % of the time, which by part (a) no controller can avoid. The replay starts the board at 70 °C plus the room's offset, so at +10 and +15 °C every run starts above the guard (the peaks there, 80.0 and 85.0 °C, are the start). The bound of part (b) holds on all 20 runs; the replay's radio model makes it loose (84.0–85.9 °C). The cost is freshness, 86.1 % at +10 °C against 98.7 % without the guard. Streaming AP changes by +0.0001 ± 0.0002 there, and by −0.0035 ± 0.0003 at +15 °C.

The hardware partly agrees. The 2 24-minute Wi-Fi missions of v4.1 (the replay's scenario, with total blackouts) spent 19–27 % of the time at or above 80 °C. In the first, 18 % of the price updates were held, and 99 % of its hot time came within 150 s (about one thermal time constant) of a held update, against 47 % of all its time. In the second, only 2 % were held, yet it was hot 27 % of the time, so its heat has another cause. The 5 24-minute mesh missions, whose prices were almost never held, spent at most 0.1 % of the time at or above 80 °C. The budget rule therefore sets the mean temperature, as stated. On long missions with blackouts, the hold explains the hot time in the replay and in one of the two hardware runs. Capping the hold at the guard's temperature removes that time in the replay; it has not yet been run on the board.

VI. Method

Strategies are compared in paired campaigns: every repetition runs all strategies on the same scenario, in shuffled order, with the same scripted link faults, following common advice on benchmarking with repetitions [38]. Differences are reported as paired means with Student-t 95 % intervals; p-values are exact sign-permutation tests, Holm-corrected over the comparisons of a campaign where stated. The board's power and temperature are measured (PMIC rails, about 3.8 samples per second); the Jetson's power is logged from its INA3221 sensor. Comparisons that cannot be run on hardware (a published planner, hindsight oracles) use a trace replay rebuilt from the recorded runs; against the 240 runs of the main campaign it reproduces per-policy streaming AP with $r = 0.957$, but it favours policies that run less onboard by about +0.0053 sAP.

VII. Results

A. Heat costs accuracy

In onboard-only runs on the Pi (YOLOv8s, YOLOv8m and YOLOv8l fine-tuned on VisDrone, with clock and throttle logging), runs of YOLOv8s that started at 60 °C ran at a mean of 84 °C with the board throttled 64 % of the time, and 89 % over 24-minute runs. Re-scoring the same runs with each result's latency at the board's full clock (an emulation), throttling cost 37–59 % of the full-clock streaming AP across the 9 detector-and-condition cells. Heat therefore costs the onboard server a large share of its accuracy, and an unpriced controller that keeps using it runs into that cost (next subsection).

B. Pricing measured power removes most of the cost

On Wi-Fi, the energy-priced controller (v4) against the same controller without prices (v3 with forecasting) gained +0.0074±0.0030 sAP, ahead in 17 of 20 pairs; a second campaign gave +0.0056±0.0037 (ahead in 18 of 20). Re-scored with the standard metric and detections of the VisDrone-trained detector, the gains are +0.0189 and +0.0165 (Holm $p = 0.002$, 0.011). On the mesh (9 pairs over three scenarios), removing the prices from v4.1 left streaming AP almost unchanged (v4.1 ahead by +0.0021, $p = 0.027$) but cost 1.47 W and 13.8 °C on the camera drone ($p = 0.004$, every pair), and the board throttled 39 % of the time against 0 % with prices. Over 24-minute mesh missions v4.1 held 0.178 sAP with the board throttled 0 % of the time (5 runs). The mechanism, less low-value onboard inference and therefore less heat and CPU contention, is consistent with these measurements but was not isolated by an ablation.

The whole system. These gains are for the camera drone. Counting the power of every board, including the edge drone that carries the Jetson, fixed offloading of every frame to the ground is the most accuracy-per-watt efficient strategy: 0.0160 sAP per watt against 0.0061–0.0067 for the priced controllers (a factor of 2.4–2.6), because it needs no edge drone. It holds a fresh result only 50 % of the time (the priced controllers at least 86 %) and fails when its link fails, so the edge drone's case is robustness, not energy.

C. Near the best schedule

Distance from the best schedule (trace replay). Knowing the exact goodput of every next transfer in advance changes v4.1's streaming AP by at most 0.0012 (inside the interval in every scenario), so a better link predictor would not help under the emulated faults. Against a per-phase hindsight oracle that picks, in each network phase, the best of v4.1 and 17 fixed settings, v4.1 reaches 97–100 % of the oracle's accuracy with 20 s phases and 88 % when the faults change every 4 s. The oracle contains v4.1, so this bounds what a better per-phase choice could add.

Published schedulers. In the 240-run hardware campaign the re-implemented literature schedulers (confidence trigger [13], offloading reward [14], freshness threshold, shortest queue) trailed the priced controller (v5) by 0.0112–0.0278 sAP (20 pairs each), and a LEAF-style fixed weight [18] tied it (−0.0004). We re-implemented an AccuMO-style planner in the trace replay: one setting per server for the next 30 frames (252 plans), the content-conditioned decay, the plan maximising the accuracy of the result on screen. Against the priced controller it was +0.0018±0.0019 on the mission (15 seeds), +0.0044±0.0019 on random link faults and +0.0014±0.0008 over 24-minute missions; the LEAF-style weight was ahead by as much (+0.0022, +0.0053, +0.0022). On fresh seeds with a corrected onboard power model the random-link difference was +0.0045±0.0026. The planner and the LEAF-style weight both draw less power than v4.1 (the planner 0.33–0.53 W less): they run less onboard. That is the direction the replay is known to favour; with the LEAF-style weight, the replay overstates the difference measured on hardware by +0.0053. Their leads, including the significant ones, are within that bias, so the supported claim is that the greedy priced choice does about as well as a 30-frame planner here (the board session tests this); the planner takes 0.64 ms per frame on a laptop CPU, and its hardware comparison is future work.

A learned policy. We also trained a constrained deep-RL policy in the replay: PPO with a Lagrange multiplier on the 2.8 W budget (PPO-Lagrangian). It was trained on 2-minute episodes from seeds not used for evaluation, with v4.1's actions and a reward for fresh, accurate results. The predictions were committed before training. Over 24-minute missions it reached 0.036 streaming AP, against v4.1's 0.046 (paired difference −0.0100 ± 0.0014). On 2-minute missions the difference was −0.0124 ± 0.0039, and with random link faults −0.0038 ± 0.0034. It drew 2.43 W against v4.1's 3.41 W. It keeps the budget, which v4.1 exceeds through its hold and cap (Theorem 1), so the comparison is at the same budget, not at the same power. Two earlier training runs failed and are reported with it: one under-charged the radio's energy, and in the other the multiplier wound up until the policy only skipped.

Calibration (trace replay). Over 44,524 offloaded frames, v4.1 predicted on average a 94.4 % chance of meeting the deadline and 98.5 % met it (expected calibration error 0.042); no probability bin was over-confident, so the controller is

TABLE II
HEADLINE PAIRED COMPARISONS (STREAMING AP WITH FORECASTER; DIFFERENCE ± 95 % INTERVAL; POSITIVE = THE FIRST STRATEGY IS BETTER). CAMERA: MEASURED CHANGE OF THE CAMERA DRONE'S POWER AND TEMPERATURE.

Comparison	Where	Δ sAP	Pairs	Camera
v4 − v3+fc (no price)	Wi-Fi	+0.0074±0.0030	20	
v4 − v3+fc, replication	Wi-Fi	+0.0056±0.0037	20	
v4.1 − no prices	mesh	+0.0021±0.0016	9	−1.47 W, −13.8 °C
heat price alone − v4.1	mesh	−0.0002±0.0022	9	+0.46 W, +4.8 °C
v3+fc − v2 (rate)	Wi-Fi	+0.0045±0.0038	20	
v5, decay without fc − v5	Wi-Fi	−0.0059±0.0042	20	
v5, flat decay − v5	Wi-Fi	−0.0054±0.0034	20	
camera decides − chain, direct	mesh	+0.0471±0.0035	6	
camera decides − chain, multihop	mesh	−0.0133±0.0057	3	
AccuMO-style − v4.1, random faults	replay	+0.0044±0.0019	15†	

†seeds of the trace replay, not hardware pairs.

cautious rather than optimistic.

D. The objective must match the forecaster

With the decay curve after forecasting, the streaming objective beat the rate objective of v2 by +0.0045±0.0038 sAP (20 pairs). In the 240-run main campaign the priced controller (v5) with the curve measured without forecasting lost 0.0059±0.0042, and with a flat curve 0.0054±0.0034; generic linear and exponential decays on the deadline's time scale tied (+0.0005, −0.0002). What matters is a decay as slow as the forecaster makes it, not the exact measured shape.

E. Where the decision belongs

On the mesh with a direct camera-to-ground link, the camera deciding among all tiers (v4.1) reached 0.193 sAP against 0.146 for a hierarchical chain in which the edge drone decides (6 pairs, $p = 0.031$). With the direct link blocked so that every packet crosses the edge drone, the order reversed: 0.151 for the chain against 0.137 for v4.1 (3 pairs).

F. Several camera drones

With two real camera drones (the Pi and the Jetson as cameras) sharing the ground GPU, sharing cost the Pi 11–19 % of its streaming AP, and adaptive control on both cameras beat fixed offloading by +0.0090±0.0062 (6 of 6 pairs). A ground-side admission coordinator rarely acted with two cameras (+0.0011±0.0027). In a swarm simulation calibrated with the measured boards and links, 20 Pi camera drones on one Wi-Fi access point each delivered 0.00 detections/s when every drone adapted alone and 4.15 with the coordinator; larger hardware swarms are future work. The same simulation tested relaying over an imperfect relay link. Each drone could send through a neighbour, chosen by its uplink, over one or two air-to-air hops. The relay links had 3, 6 or 8 dB of shadowing and 0–10 % per-packet loss with retransmission. There were 144 settings with 5–20 drones and 5 seeds each. The adaptive policy relayed at most 1.9 % of the camera frames. Whether a relayed frame arrived on time depended on the swarm, not on the relay link: from 0 % to 100 % across channel profiles and swarm sizes, and within each of them the noise moved the share by at most 8 points. Relaying changed the adaptive policy's rate by −0.36 to 0.37 detections/s per drone, at most 4 % wherever it delivered at least one per second. The shared access point, not the relay link, limits the swarm.

G. Negative results

A predictive heat price added nothing on top of the energy price: on Wi-Fi, v5 (both prices) against v4.2 (energy price only) changed streaming AP by +0.0018±0.0024 and the camera drone's power by −0.01±0.06 W (20 pairs). In place of the energy price it did worse: on the mesh, the heat price alone kept the same accuracy as v4.1 (−0.0002) at 0.46 W and 4.8 °C more on the camera drone ($p = 0.004$), because it stays at zero until the predicted temperature reaches 75 °C, while the energy price acts on every joule. Pricing the measured power is the better signal. Pricing the Jetson's energy at a 0.5 W budget changed streaming AP by −0.0156, and at 1.2 W by −0.0023 ($p = 0.043$), for little saving, because the edge drone's power is dominated by its idle draw.

VIII. LIMITATIONS

The boards were on a table, one hop apart, with recorded video replayed; the mesh runs batman-adv as is, one of several routing protocols compared for UAV meshes [39], and its own effect was not characterised; there was one camera drone on the mesh; most mesh cells have 3 repetitions; the trace replay favours low-onboard policies by about +0.0053 sAP, so the planner comparison is provisional; the throttling cost is measured against an emulated full clock, and the energy mechanism is not isolated by an ablation; whether the controller also survives correlated failures of both links under real interference is not yet measured; the step-size, budget, hold and room-temperature results come from the replay, whose thermal plant is the model of Proposition 1, and the room temperature was not logged, so the capped hold still needs a board run.

IX. CONCLUSION

On a fanless drone computer heat is a hidden accuracy cost: onboard detection keeps the board throttled, and throttling

alone costs a large share of onboard streaming accuracy. A greedy scheduler that values each frame by the accuracy its result will still have, and prices the camera drone's measured power and the freshness of its results, removes most of that cost while keeping or raising streaming accuracy; on long missions with total blackouts its blackout hold still lets the board run hot, which a hold capped at the guard's temperature removes in the replay. In a validated replay it reaches 97–100 % of a per-phase hindsight oracle. Its power budget holds on every run up to a price term and a hold-and-cap term, and it sets the board's mean temperature. Its objective must use the decay curve of the forecaster in use.

REFERENCES

[1] M.-R. Ra, A. Sheth, L. Mummert, P. Pillai, D. Wetherall, and R. Govindan, "Odessa: enabling interactive perception applications on mobile devices," in *Proceedings of the 9th international conference on Mobile systems, applications, and services*, ser. MobiSys'11. ACM, Jun. 2011, pp. 43–56. [Online]. Available: http://dx.doi.org/10.1145/1999995.2000000

[2] T. Y.-H. Chen, L. Ravindranath, S. Deng, P. Bahl, and H. Balakrishnan, "Glimpse: Continuous, real-time object recognition on mobile devices," in *Proceedings of the 13th ACM Conference on Embedded Networked Sensor Systems*, ser. SenSys '15. ACM, Nov. 2015, pp. 155–168. [Online]. Available: http://dx.doi.org/10.1145/2809695.2809711

[3] W. Zhang, Z. He, L. Liu, Z. Jia, Y. Liu, M. Gruteser, D. Raychaudhuri, and Y. Zhang, "Elf: accelerate high-resolution mobile deep vision with content-aware parallel offloading," in *Proceedings of the 27th Annual International Conference on Mobile Computing and Networking*, ser. ACM MobiCom '21. ACM, Sep. 2021, pp. 201–214. [Online]. Available: http://dx.doi.org/10.1145/3447993.3448628

[4] M. Li, Y.-X. Wang, and D. Ramanan, *Towards Streaming Perception*. Springer International Publishing, 2020, pp. 473–488. [Online]. Available: http://dx.doi.org/10.1007/978-3-030-58536-5_28

[5] J. Yang, S. Liu, Z. Li, X. Li, and J. Sun, "Real-time object detection for streaming perception," in *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, Jun. 2022, pp. 5375–5385. [Online]. Available: http://dx.doi.org/10.1109/cvpr52688.2022.00531

[6] A. Coletta, F. Giorgi, G. Maselli, M. Prata, D. Silvestri, J. Ashdown, and F. Restuccia, "A^2-UAV: Application-aware content and network optimization of edge-assisted uav systems," in *IEEE INFOCOM 2023 - IEEE Conference on Computer Communications*. IEEE, May 2023, pp. 1–10. [Online]. Available: http://dx.doi.org/10.1109/infocom53939.2023.10229096

[7] S. Jeon, J. Kim, J. Lee, and H. Cha, "Phoenix: Thermal-aware on-device inference of multi-instance dnns for mobile video applications," *ACM Transactions on Embedded Computing Systems*, vol. 25, no. 2, pp. 1–23, Mar. 2026. [Online]. Available: http://dx.doi.org/10.1145/3793860

[8] Z. J. Kong, Q. Xu, J. Meng, and Y. C. Hu, "Accumo: Accuracy-centric multitask offloading in edge-assisted mobile augmented reality," in *Proceedings of the 29th Annual International Conference on Mobile Computing and Networking*, ser. ACM MobiCom '23. ACM, Oct. 2023, pp. 1–16. [Online]. Available: http://dx.doi.org/10.1145/3570361.3592531

[9] J. Jiang, G. Ananthanarayanan, P. Bodik, S. Sen, and I. Stoica, "Chameleon: scalable adaptation of video analytics," in *Proceedings of the 2018 Conference of the ACM Special Interest Group on Data Communication*, ser. SIGCOMM '18. ACM, Aug. 2018, pp. 253–266. [Online]. Available: http://dx.doi.org/10.1145/3230543.3230574

[10] B. Zhang, X. Jin, S. Ratnasamy, J. Wawrzynek, and E. A. Lee, "Awstream: adaptive wide-area streaming analytics," in *Proceedings of the 2018 Conference of the ACM Special Interest Group on Data Communication*, ser. SIGCOMM '18. ACM, Aug. 2018, pp. 236–252. [Online]. Available: http://dx.doi.org/10.1145/3230543.3230554

[11] S. Zhang, C. Wang, Y. Jin, J. Wu, Z. Qian, M. Xiao, and S. Lu, "Adaptive configuration selection and bandwidth allocation for edge-based video analytics," *IEEE/ACM Transactions on Networking*, vol. 30, no. 1, pp. 285–298, Feb. 2022. [Online]. Available: http://dx.doi.org/10.1109/tnet.2021.3106937

[12] K. Du, A. Pervaiz, X. Yuan, A. Chowdhery, Q. Zhang, H. Hoffmann, and J. Jiang, "Server-driven video streaming for deep learning inference," in *Proceedings of the Annual conference of the ACM Special Interest Group on Data Communication on the applications, technologies, architectures, and protocols for computer communication*, ser. SIGCOMM '20. ACM, Jul. 2020, pp. 557–570. [Online]. Available: http://dx.doi.org/10.1145/3387514.3405887

[13] C. Gomez, D. Patkhullaev, and A. C. Hernandez, "Offload: Detection failure-based scheduler for offloading object detection," *ACM Transactions on Embedded Computing Systems*, vol. 23, no. 5, pp. 1–26, Aug. 2024. [Online]. Available: http://dx.doi.org/10.1145/3677321

[14] J. Qiu, R. Wang, B. Hu, R. Guérin, and C. Lu, "Optimizing edge offloading decisions for object detection," in *2024 IEEE/ACM Symposium on Edge Computing (SEC)*. IEEE, Dec. 2024, pp. 164–177. [Online]. Available: http://dx.doi.org/10.1109/sec62691.2024.00021

[15] A. Ghosh, V. Balloli, A. Nambi, A. Singh, and T. Ganu, "Chanakya: Learning runtime decisions for adaptive real-time perception," in *Advances in Neural Information Processing Systems 36*, ser. NeurIPS 2023. Neural Information Processing Systems Foundation, Inc. (NeurIPS), 2023, pp. 55 668–55 680. [Online]. Available: http://dx.doi.org/10.52202/075280-2429

[16] T. Wang, L. Zhu, and H. Huang, "Enhancing real-time object detection with optical flow-guided streaming perception," *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 35, no. 5, pp. 4816–4830, May 2025. [Online]. Available: http://dx.doi.org/10.1109/tcsvt.2025.3525796

[17] X. Li, S. Zhang, Y. Huang, X. Ma, Z. Wang, and H. Luo, "Towards timely video analytics services at the network edge," *IEEE Transactions on Mobile Computing*, vol. 23, no. 11, pp. 10 443–10 459, Nov. 2024. [Online]. Available: http://dx.doi.org/10.1109/tmc.2024.3376769

[18] H. Wang, B. Kim, J. Xie, and Z. Han, "Leaf + aio: Edge-assisted energy-aware object detection for mobile augmented reality," *IEEE Transactions on Mobile Computing*, vol. 22, no. 10, pp. 5933–5948, Oct. 2023. [Online]. Available: http://dx.doi.org/10.1109/tmc.2022.3179943

[19] Z. Xu, L. Zhao, W. Liang, O. F. Rana, P. Zhou, Q. Xia, W. Xu, and G. Wu, "Energy-aware inference offloading for dnn-driven applications in mobile edge clouds," *IEEE Transactions on Parallel and Distributed Systems*, vol. 32, no. 4, pp. 799–814, Apr. 2021. [Online]. Available: http://dx.doi.org/10.1109/tpds.2020.3032443

[20] H. Jiang, X. Dai, Z. Xiao, and A. Iyengar, "Joint task offloading and resource allocation for energy-constrained mobile edge computing," *IEEE Transactions on Mobile Computing*, vol. 22, no. 7, pp. 4000–4015, Jul. 2023. [Online]. Available: http://dx.doi.org/10.1109/tmc.2022.3150432

[21] A. Younis, S. Maheshwari, and D. Pompili, "Energy-latency computation offloading and approximate computing in mobile-edge computing networks," *IEEE Transactions on Network and Service Management*, vol. 21, no. 3, pp. 3401–3415, Jun. 2024. [Online]. Available: http://dx.doi.org/10.1109/tnsm.2024.3360850

[22] A. M. Zaki, S. A. Elsayed, K. Elgazzar, and H. S. Hassanein, "Quality-aware task offloading for cooperative perception in vehicular edge computing," *IEEE Transactions on Vehicular Technology*, vol. 73, no. 12, pp. 18 320–18 332, Dec. 2024. [Online]. Available: http://dx.doi.org/10.1109/tvt.2024.3444591

[23] T. Benoit-Cattin, D. Velasco-Montero, and J. Fernández-Berni, "Impact of thermal throttling on long-term visual inference in a CPU-based edge device," *Electronics*, vol. 9, no. 12, p. 2106, 2020.

[24] T. Tan and G. Cao, "Thermal-aware scheduling for deep learning on mobile devices with NPU," *IEEE Transactions on Mobile Computing*, vol. 23, no. 12, pp. 10 706–10 719, 2024.

[25] P. Choi, J. Kim, and J. Kwak, "Impact of joint heat and memory constraints of mobile device in edge-assisted on-device artificial intelligence," in *Proceedings of the 2nd International Workshop on Networked AI Systems (MobiSys '24)*. ACM, 2024, pp. 31–36.

[26] H. Zhao, G. Lu, Y. Liu, Z. Chang, L. Wang, and T. Hämäläinen, "Safe DQN-based AoI-minimal task offloading for UAV-aided edge computing system," *IEEE Internet of Things Journal*, vol. 11, no. 19, pp. 32 012–32 024, 2024.

[27] B. Boroujerdian, H. Genc, S. Krishnan, W. Cui, A. Faust, and V. Reddi, "MAVBench: Micro aerial vehicle benchmarking," in *2018 51st Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2018, pp. 894–907.

[28] C. Qu, R. Singh, A. Esquivel-Morel, and P. Calyam, "Learning-based multi-drone network edge orchestration for video analytics,"

IEEE Transactions on Network and Service Management, vol. 21, no. 6, pp. 6331–6348, Dec. 2024. [Online]. Available: http://dx.doi.org/10.1109/tnsm.2024.3440883

[29] G. Sun, Y. Wang, Z. Sun, Q. Wu, J. Kang, D. Niyato, and V. C. M. Leung, "Multi-objective optimization for multi-uav-assisted mobile edge computing," *IEEE Transactions on Mobile Computing*, vol. 23, no. 12, pp. 14 803–14 820, Dec. 2024. [Online]. Available: http://dx.doi.org/10.1109/tmc.2024.3446819

[30] H. Hao, C. Xu, W. Zhang, S. Yang, and G.-M. Muntean, "Joint task offloading, resource allocation, and trajectory design for multi-uav cooperative edge computing with task priority," *IEEE Transactions on Mobile Computing*, vol. 23, no. 9, pp. 8649–8663, Sep. 2024. [Online]. Available: http://dx.doi.org/10.1109/tmc.2024.3350078

[31] Y. Kang, J. Hauswald, C. Gao, A. Rovinski, T. Mudge, J. Mars, and L. Tang, "Neurosurgeon: Collaborative intelligence between the cloud and mobile edge," in *Proc. 22nd Int. Conf. Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2017, pp. 615–629.

[32] Y. Matsubara, M. Levorato, and F. Restuccia, "Split computing and early exiting for deep learning applications: Survey and research challenges," *ACM Computing Surveys*, vol. 55, no. 5, pp. 1–30, 2023.

[33] Y. Matsubara, R. Yang, M. Levorato, and S. Mandt, "Supervised compression for resource-constrained edge computing systems," in *2022 IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*. IEEE, Jan. 2022, pp. 923–933. [Online]. Available: http://dx.doi.org/10.1109/wacv51458.2022.00100

[34] A. Furutanpey, P. Raith, and S. Dustdar, "Frankensplit: Efficient neural feature compression with shallow variational bottleneck injection for mobile edge computing," *IEEE Transactions on Mobile Computing*, vol. 23, no. 12, pp. 10 770–10 786, Dec. 2024. [Online]. Available: http://dx.doi.org/10.1109/tmc.2024.3381952

[35] P. Zhu, L. Wen, D. Du, X. Bian, H. Fan, Q. Hu, and H. Ling, "Detection and tracking meet drones challenge," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 11, pp. 7380–7399, 2022.

[36] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLOv8," 2023, version 8.0.0. https://github.com/ultralytics/ultralytics.

[37] M. J. Neely, *Stochastic Network Optimization with Application to Communication and Queueing Systems*, ser. Synthesis Lectures on Learning, Networks, and Algorithms. Springer International Publishing, 2010. [Online]. Available: http://dx.doi.org/10.1007/978-3-031-79995-2

[38] T. Kalibera and R. Jones, "Rigorous benchmarking in reasonable time," in *Proceedings of the 2013 international symposium on memory management*, ser. ISMM '13. ACM, Jun. 2013, pp. 63–74. [Online]. Available: http://dx.doi.org/10.1145/2464157.2464160

[39] B. A. Diniz, I. G. Ferrã, L. M. d. Silva, and K. C. Branco, "Comparative performance analysis of olsr, batman-adv, and babel in uav mesh networks," in *2025 International Conference on Unmanned Aircraft Systems (ICUAS)*. IEEE, May 2025, pp. 496–503. [Online]. Available: http://dx.doi.org/10.1109/icuas65942.2025.11007820